

1. Find antal samples N for alle tre signaler.

Vi starter ud med at hente dataen fra de 3 lydfile, hvor vi derudaf får variablerne: data (1,2,3) fs(1,2,3) - altså samplingsfrekvenserne på de 3 signaler.

Det gøres således i MatLab:

```
>> [data1, fs1] = audioread("Signal_s1.mp3");  
>> [data2, fs2] = audioread("Signal_s2.mp3");  
>> [data3, fs3] = audioread("Signal_s3.mp3");
```

Jeg kan nu se følgende data:

data1	4585584×2 double	4585584×2	double
data2	9526512×2 double	9526512×2	double
data3	1381872×2 double	1381872×2	double
fs1	48000	1×1	double
fs2	48000	1×1	double
fs3	48000	1×1	double

Jeg undrer mig lidt over de alle bliver kaldt for double når nu s_1 og s_3 er monosignaler. Efter lidt research fandt ud af, det er fordi Brightspace ikke tillader at downloade wav filer, og signalerne er downloadet som mp3 i stedet.

Vi bruger funktionen "size" i MatLab for at finde frem til N for hvert signal:

```
>> N1 = size(data1,1)  
N2 = size(data2,1)  
N3 = size(data3,1)
```

```
N1 =  
  
4585584
```

```
N2 =  
  
9526512
```

```
N3 =  
  
1381872
```

Hvor jeg har sat (data1,1) ind så jeg kan regne videre med N .

Her kan vi se at N er:

- Signal 1: 4.585.584
- Signal 2: 9.526.512

- Signal 3: 1.381.872
-

2. Plot alle tre signaler med korrekte tidsakser, og sæt `xlabel`, `ylabel` og `title` på.

For at lave plot alle tre signaler, skal vi først beregne sampletælleren(n) og tidsaksen(t) for hver af de tre signaler. Jeg gør således i MatLab:

```
>> n1 = [0:N1-1];  
>> n2 = [0:N2-1];  
>> n3 = [0:N3-1];  
>> t1 = n1/fs1;  
>> t2 = n2/fs2;  
>> t3 = n3/fs3;
```

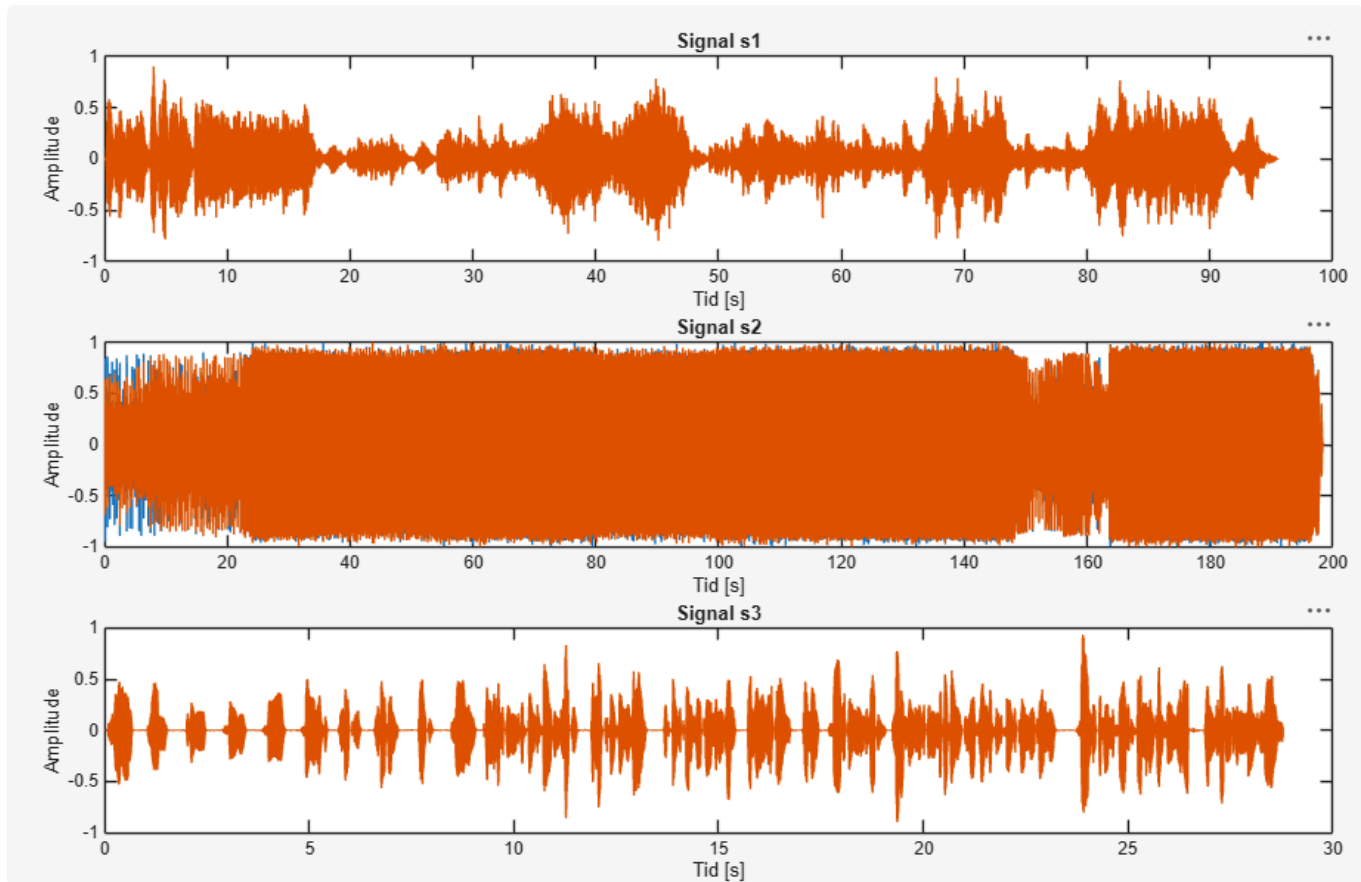
Nu kan jeg så lave mine plots. Jeg laver dem som subplots så jeg kan se alle 3 samtidigt. Jeg skriver det op således i MatLab:

```
>> subplot(3,1,1)  
plot(t1, data1)  
xlabel('Tid [s]')  
ylabel('Amplitude')  
title('Signal s1')
```

```
subplot(3,1,2)  
plot(t2, data2)  
xlabel('Tid [s]')  
ylabel('Amplitude')  
title('Signal s2')
```

```
subplot(3,1,3)  
plot(t3, data3)  
xlabel('Tid [s]')  
ylabel('Amplitude')  
title('Signal s3')
```

Og får disse plots:



3. Find for alle tre signaler: max, min, gennemsnit (mean), rms og effekt.

Jeg bruger MatLabs funktioner:

- `max(data)`
- `min(data)`
- `mean(data)`
- `rms(data)`

Signal 1

Regner jeg det ud således i MatLab for signal 1, som er et monosignal:

```
MAX1 = max(data1(:,1))
MIN1 = min(data1(:,1))
MEAN1 = mean(data1(:,1))
RMS1 = rms(data1(:,1))
EFFEKT1 = RMS1^2
```

Her er det igen værd at nævne, at matlab har indlæst signalet med 2 kanaler, så det er nødvendigt at skrive `(:,1)` ind i min funktion

Og får resultaterne:

MAX1 =

0.9035

MIN1 =

-0.7979

MEAN1 =

-4.0966e-05

RMS1 =

0.1163

EFFEKT1 =

0.0135

Signal 1	
Max	0.9035
Min	-0.7979
Gennemsnit	-0.000041
rms	0.1163
Effekt	0.0135

Analyse

Max og min (0,9035 og $-0,7979$) er tæt på grænserne for et normaliseret lydsignal, hvilket fortæller os at der er meget høje udsving i signalet.||

Middelværdien er meget tæt på 0 ($-4,0966 \times 10^{-5}$), hvilket er normalt for et lydsignal, da lyd svinger symmetrisk omkring 0. Havde middelværdien i stedet ligget langt fra 0, ville det betyde et DC-offset — en forskydning af signalet væk fra 0-linjen, typisk pga. fejl i optageudstyret.

RMS (0,1163) er markant lavere end max. Det giver god mening, da vi kan se på vores plot, at signalet oftest ligger et godt stykke under max og veksler meget mellem stille og kraftige passager. Effekten (0,0135) bekræfter samme billede.

Signal 2

Signal 2 er et stereo-signal med 2 kanaler. Derfor skal jeg finde max, min, gennemsnit (mean), rms og effekt for begge kanaler. Det gør jeg på samme måde som for signal 1:

```
MAX2_1 = max(data2(:,1))
MAX2_2 = max(data2(:,2))
MIN2_1 = min(data2(:,1))
MIN2_2 = min(data2(:,2))
MEAN2_1 = mean(data2(:,1))
MEAN2_2 = mean(data2(:,2))
RMS2_1 = rms(data2(:,1))
RMS2_2 = rms(data2(:,2))
EFFEKT2_1 = RMS2_1^2
EFFEKT2_2 = RMS2_2^2
```

Dette er mine resultater:

Signal 2	Kanal 1	Kanal 2
Max	1	1
Min	-1	-1
Gennemsnit	-0,0000053	-0,0000154
rms	0,3285	0,3445
Effekt	0,1079	0,1187

Analyse

Max og min rammer præcis grænserne for et normaliseret lydsignal (± 1) på begge kanaler. Det skyldes sandsynligvis kompression i mastering-processen, hvor moderne, "loud" produceret musik bevidst presses helt op mod grænsen for at opleves konsekvent kraftig.

Gennemsnittet ligger utroligt tæt på 0 for begge kanaler, hvilket er normalt for et lydsignal uden DC-offset.

RMS er markant højere end for s_1 (0,33-0,34 mod 0,1163), hvilket betyder at signalet generelt ligger tættere på max. Det kan vi også se i plottet, hvor s_2 ser væsentligt mere komprimeret ud end s_1 - mindre forskel mellem stille og kraftige passager.

Kanal 2 er svagt højere end kanal 1, men det er en meget lille forskel.

Signal 3

For signal 3 gør vi præcis det samme som vi gjorde for signal 1:

```
MAX3 = max(data3(:,1))
MIN3 = min(data3(:,1))
MEAN3 = mean(data3(:,1))
RMS3 = rms(data3(:,1))
EFFEKT3 = RMS3^2
```

Vi får resultaterne:

Signal 3	
Max	0,9335
Min	-0,8973
Gennemsnit	-0,0006
rms	0,0892
Effekt	0,0080

Analyse

Max og min minder meget om s_1 's naturlige udsving.

Gennemsnittet ligger meget tæt på 0, som forventet.

RMS (0,0892) er det laveste af de tre signaler. Ud fra plottet giver det god mening, da signalet langt hen ad vejen ligger et godt stykke under sit max.

4. Find de tre signalers crest-faktorer — sammenlign og kommentér.

For at finde crest faktoren bruger vi formlen (angives i decibel, da det er den måde crest-faktor typisk anvendes til at karakterisere audio-signaler på):

$$Crest-faktor = 20 \log_{10} \frac{|x_{peak}|}{x_{RMS}}$$

x_{peak} er ganske simpelt den største numeriske værdi, så vi sammenligner max vs min i det givne signal.

Signal 1

Max1 = 0.9035 Min1 = -0.7979 rms1 = 0.1163 $x_{peak} = 0.9035$

$$cf_1 = 20 \log_{10} \left(\frac{0.9035}{0.1163} \right) = 20 \log_{10}(7.77) = 17,81 \text{ dB}$$

Signal 2

Kanal 1: $\text{Max}_{2.1} = 1$ $\text{Min}_{2.1} = -1$ $\text{rms}_{2.1} = 0.3285$ $x_{\text{peak}} = 1$

$$cf_{2.1} = 20 \log_{10} \left(\frac{1}{0.3285} \right) = 20 \log_{10}(3.044) = 9,67 \text{ dB}$$

Kanal 2: $\text{Max}_{2.2} = 1$ $\text{Min}_{2.2} = -1$ $\text{rms}_{2.2} = 0.3445$ $x_{\text{peak}} = 1$

$$cf_{2.2} = 20 \log_{10} \left(\frac{1}{0.3445} \right) = 20 \log_{10}(2.903) = 9,26 \text{ dB}$$

Signal 3

$\text{Max}_3 = 0.9335$ $\text{Min}_3 = -0.8973$ $\text{rms}_3 = 0.0892$ $x_{\text{peak}} = 0.9335$

$$cf_3 = 20 \log_{10} \left(\frac{0.9335}{0.0892} \right) = 20 \log_{10}(10.465) = 20,39 \text{ dB}$$

Så samlet resultattabel til sammenligning:

Signal 1	Signal 2 kanal 1	Signal 2 kanal 2	Signal 3
17,81 dB	9,67 dB	9,26 dB	20,39 dB

Helt overordnet set betyder en høj crest faktor, at peak er meget større end RMS, og at signalet er meget dynamisk - altså at der er stor forskel mellem de stille og de kraftige passager.

Hvis vi rangerer dem, kan vi se hvilket signal der er mest dynamisk, og hvilket der er mest fladt (mest komprimeret):

1. Signal 3: 20,39 dB
2. Signal 1: 17,81 dB
3. Signal 2 kanal 1: 9,67 dB
4. Signal 2 kanal 2: 9,26 dB

Det stemmer godt overens med observationerne fra opgave 3: s_2 havde markant højere RMS relativt til sit max end de to andre signaler (loud-mastret musik presset tæt på grænsen), hvilket netop giver en lav crest-faktor, mens s_3 (oplæsning) har den højeste crest-faktor, da talesignalet veksler meget mellem stilhed/pauser og kraftige stavelser.

5. Nedsample s_1 med en faktor 4. Plot — og sammenlign med originalen.

Vi starter med at lave en ny fs for signalet, hvor vi dividerer den nuværende fs med 4 og derefter bruger vi "decimate" funktionen i MatLab til at nedsample vores data:

```
fs1_ny = fs1/4
data1_ny = decimate(data1(:,1), 4);

fs1_ny =

    12000
```

Så beregner jeg den nye N1, n1 og t1:

```
N1_ny = size(data1_ny,1)

N1_ny =

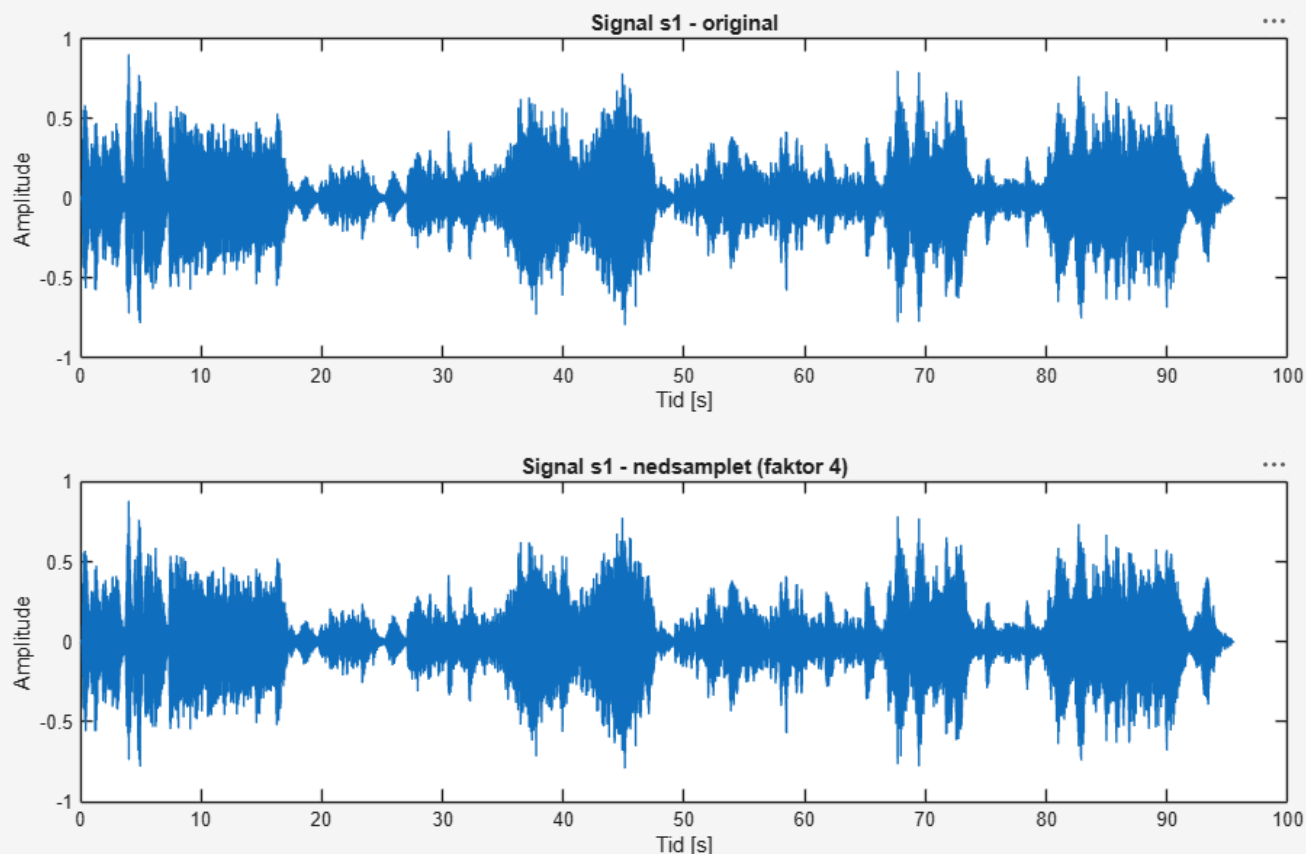
    1146396

>>
n1_ny = [0:N1_ny-1];
t1_ny = n1_ny/fs1_ny;
```

Jeg laver nu subplots med signal 1 og det nedsampled signal 1:

```
subplot(2,1,1)
plot(t1, data1(:,1))
xlabel('Tid [s]')
ylabel('Amplitude')
title('Signal s1 - original')

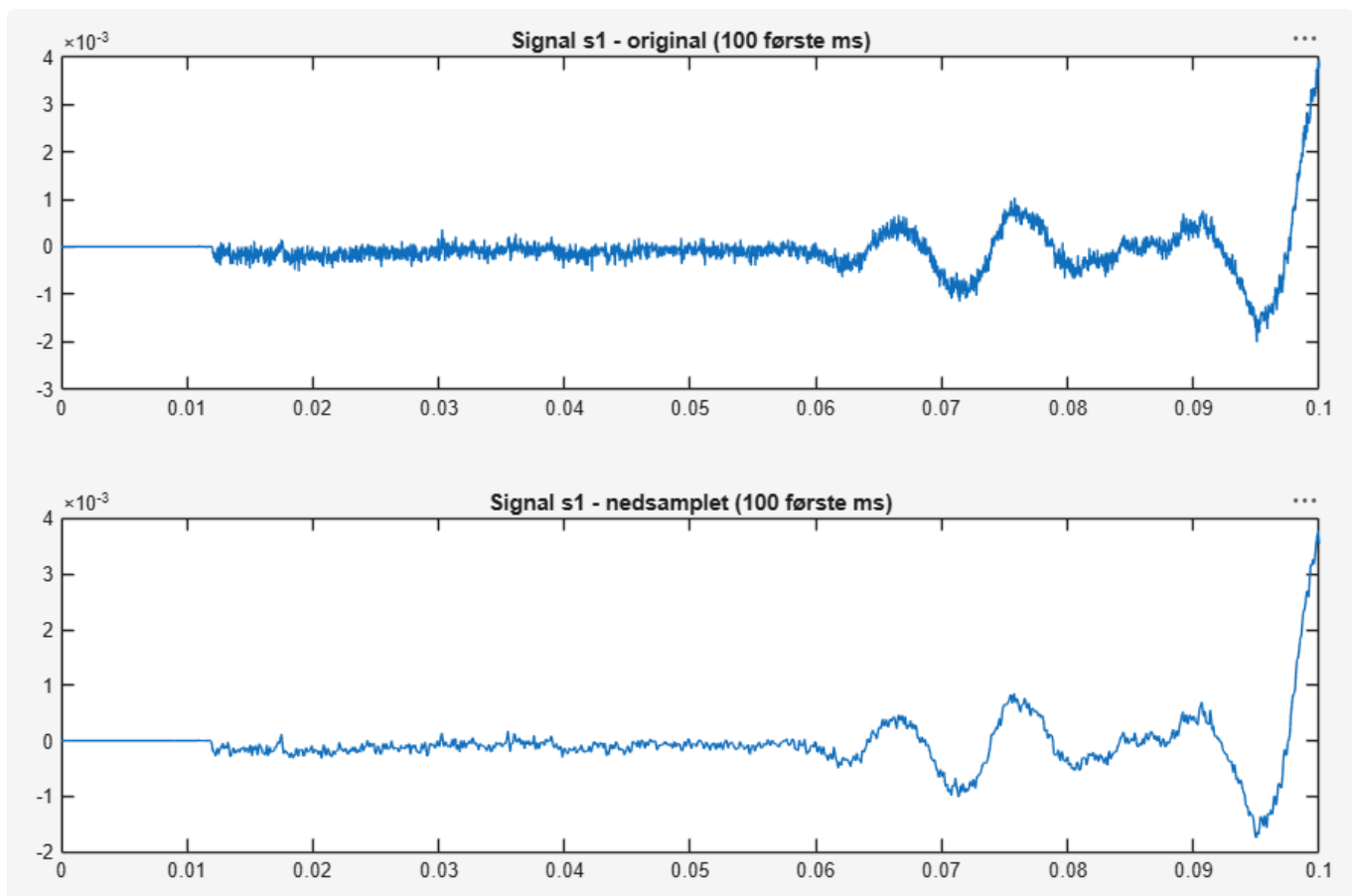
subplot(2,1,2)
plot(t1_ny, data1_ny)
xlabel('Tid [s]')
ylabel('Amplitude')
title('Signal s1 - nedsamplet (faktor 4)')
```



Signalerne ligner fuldkommen hinanden, hvilket kan have noget at gøre med at vi kigger på signalerne over 100 sekunder. Vi kan vælge at zoom ind på de første 100 millisekunder, og se om der er forskel:

```
subplot(2,1,1)
plot(t1, data1(:,1))
xlim([0 0.1])
title('Signal s1 - original (100 første ms)')

subplot(2,1,2)
plot(t1_ny, data1_ny)
xlim([0 0.1])
title('Signal s1 - nedsamplet (100 første ms)')
```



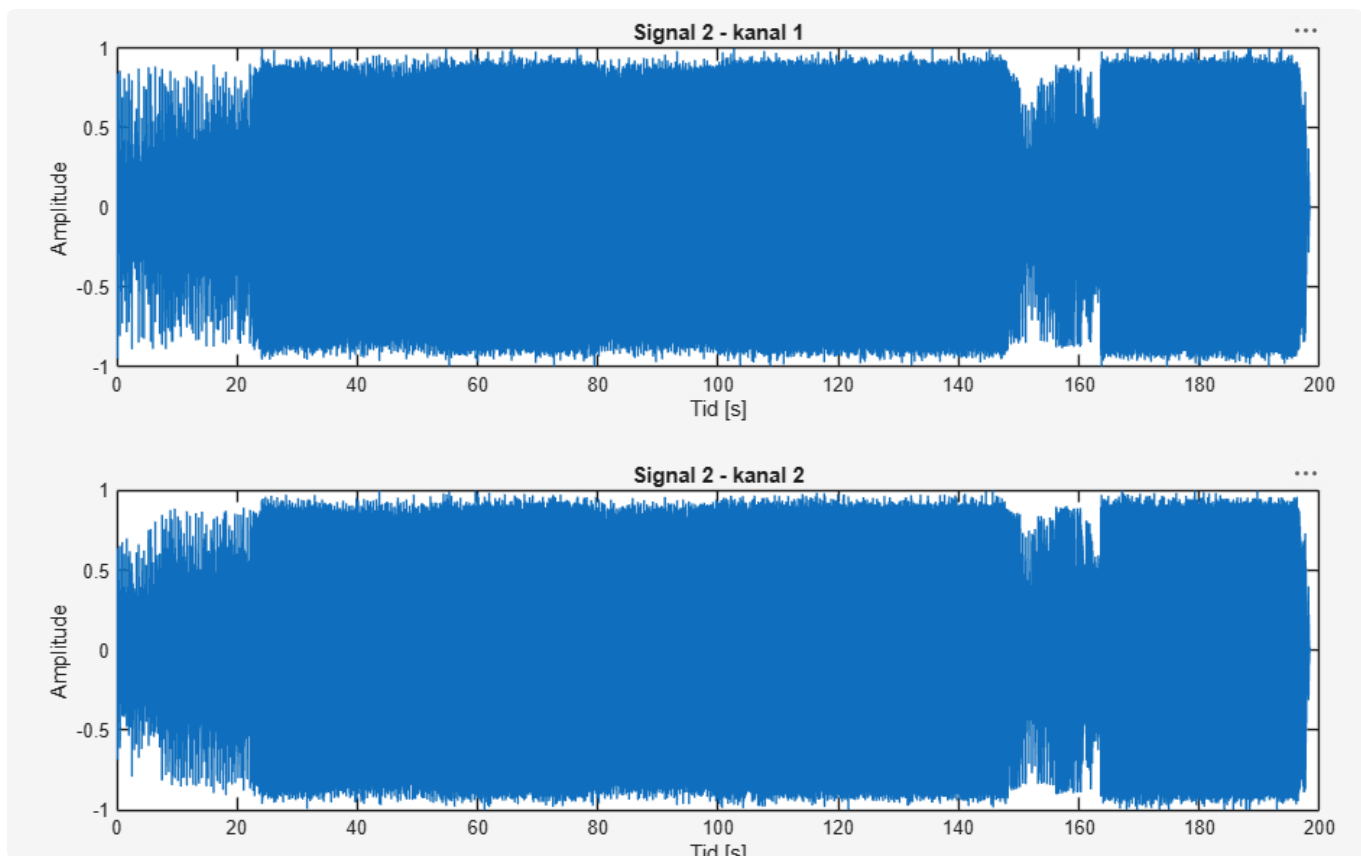
Her kan vi tydeligt se forskel. Originalen har langt flere udsving mens den nedsamlet version ser mere flad ud så og sige.

6. Lyt — og beskriv forskellen mellem originalen og den nedsamplede version.

Den nedsamplede version er langt mere utydelig, og i meget dårligere kvalitet. Man kan sige den lyder mere dæmpet. Det stemmer med det zoomede plot fra opgave 5 - decimate benytter et lavpasfilter (anti-aliasfilter), inden der nedsamples, som fjerner de høje frekvenser i signalet. Det er netop de høje frekvenser, der giver lyden dens klarhed/skarphe, så det forklarer den mere dæmpede/matte lyd.

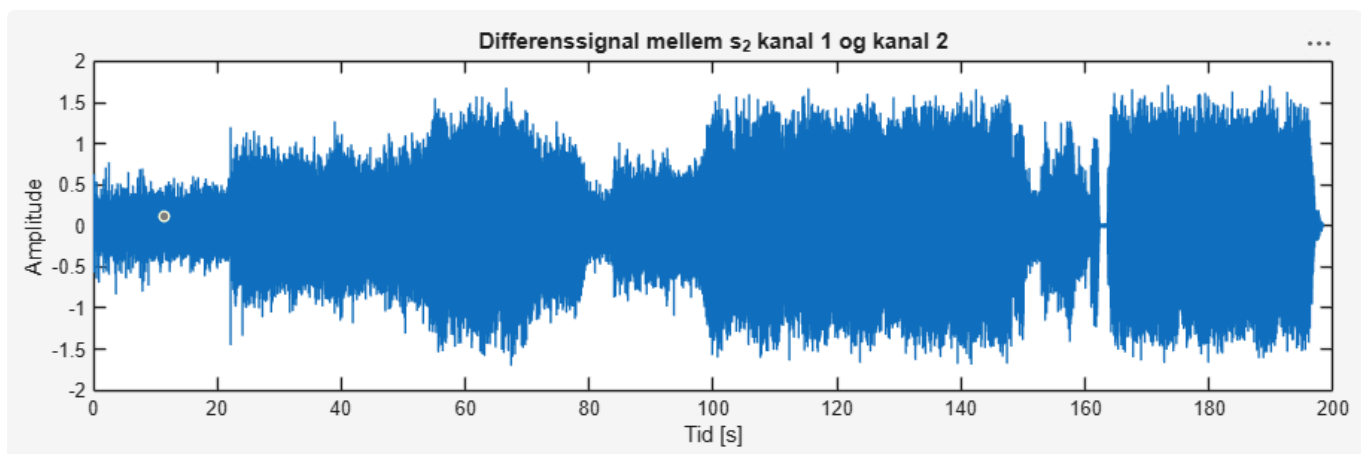
7. Sammenlign de to stereo-kanaler i s_2 — beskriv forskelle og ligheder.

De lyder super ens for at være ærlig. Jeg kan umiddelbart ikke høre nogen forskel, derfor kigger jeg på de individuelle kanaler i stedet:



Her kan vi se der er større udsving i starten på kanal et, og større udsving på kanal 2 omkring 150 sekund. Derudover er de meget ens.

For at undersøge forskellen mere præcist beregnede jeg differenssignalet mellem de to kanaler, $d(n) = s_{2,K1}(n) - s_{2,K2}(n)$:



Differenssignalet er langt fra en flad linje ved 0 - der er udsving på op til ca. $\pm 1,5-2$. Det bekræfter, at der målbart er en betydelig forskel mellem kanalerne på sample-niveau, selvom forskellen ikke var tydeligt hørbar, da jeg lyttede til kanalerne hver for sig.

8. Kvantisér s_2 (venstre kanal) med 4 bit (brug egen funktion eller den på BS). Plot og lyt — sammenlign med originalen og kommentér.

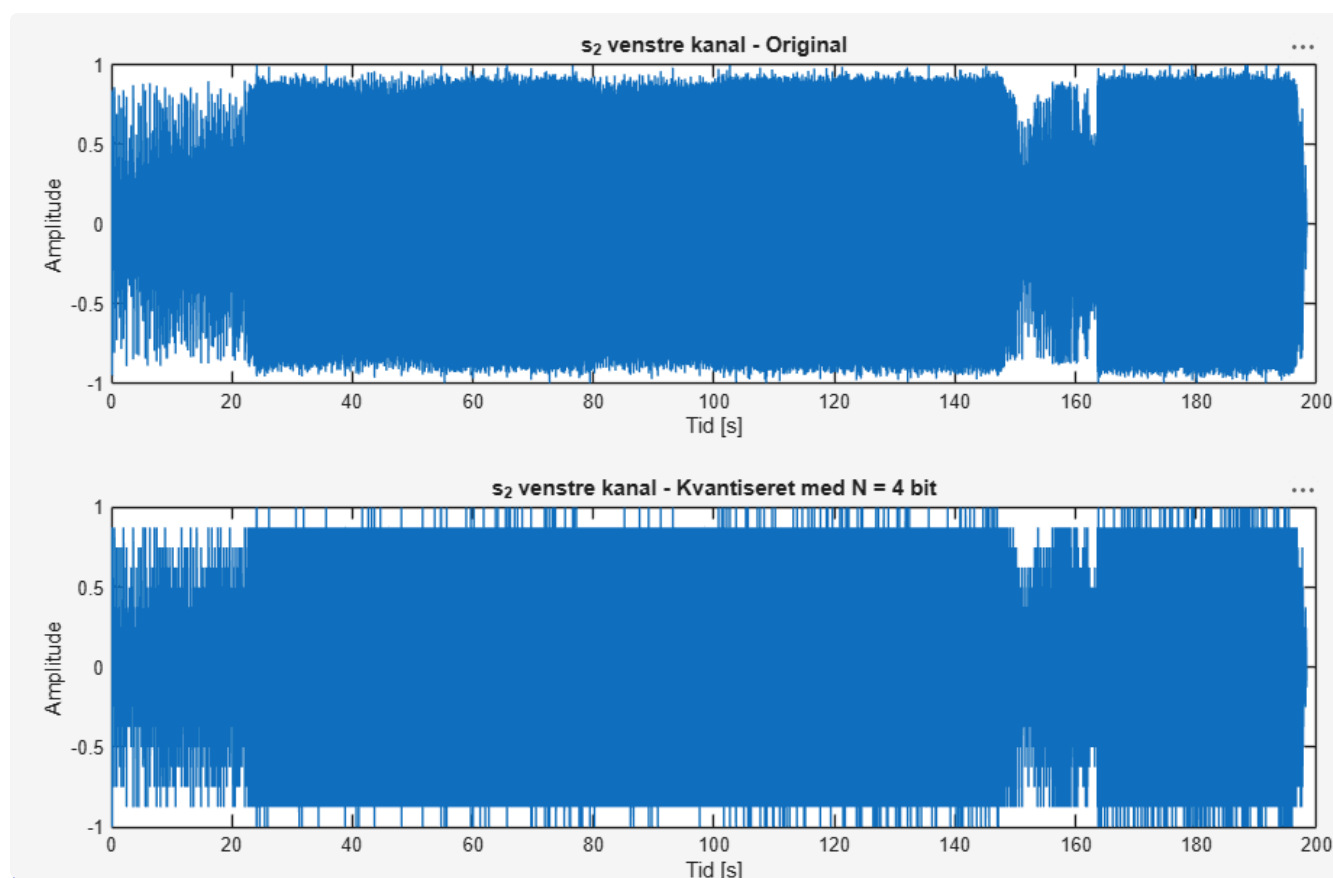
For at kvantisere signal 2 laver jeg først et script som jeg kan bruge i mit command window i MatLab:

```
quantize.m × +
C:\Users\rober\Software-Teknologi\Programmering\3. Semester\Digital signalbehandling\quantize.m
1 function xq = quantize(x, N)
2     V = max(abs(x));
3     delta = 2*V / 2^N;
4     xq = delta * round(x/delta);
5 end
```

Ovenstående funktion tager input x (signalet) og N (antal bit). Den returnerer så xq som er signalet kvantiseret.

```
x_orig = data2(:,1);
N = 4;
x_quant = quantize(x_orig, N);
```

Vi definerer så signal 2's venstre kanal som x_orig, og definerer N = 4.



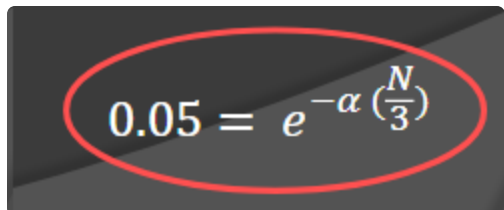
Der er tydelig forskel i lyden. Det kvantiseret signal skratter utroligt meget, mens det originale signal er langt mere rent, hvilket giver god mening eftersom hvert sample rundes af. Det kan vi også tydeligt se på vores plot.

Det stemmer også overens med teorien om signal-kvantiseringsstøj-forhold (SQNR):

$$SQNR = 6,02 \cdot N + 1,76 = 6,02 \cdot 4 + 1,76 = 25,84 \text{ dB}$$

9. Lav et "fade out" på s_3 over den sidste tredjedel med en eksponentielt aftagende envelope, hvor amplituden ved slutningen er aftaget til 5%.

Vi bruger formelen vi har fået opgivet i opgavebeskrivelsen:


$$0.05 = e^{-\alpha(\frac{N}{3})}$$

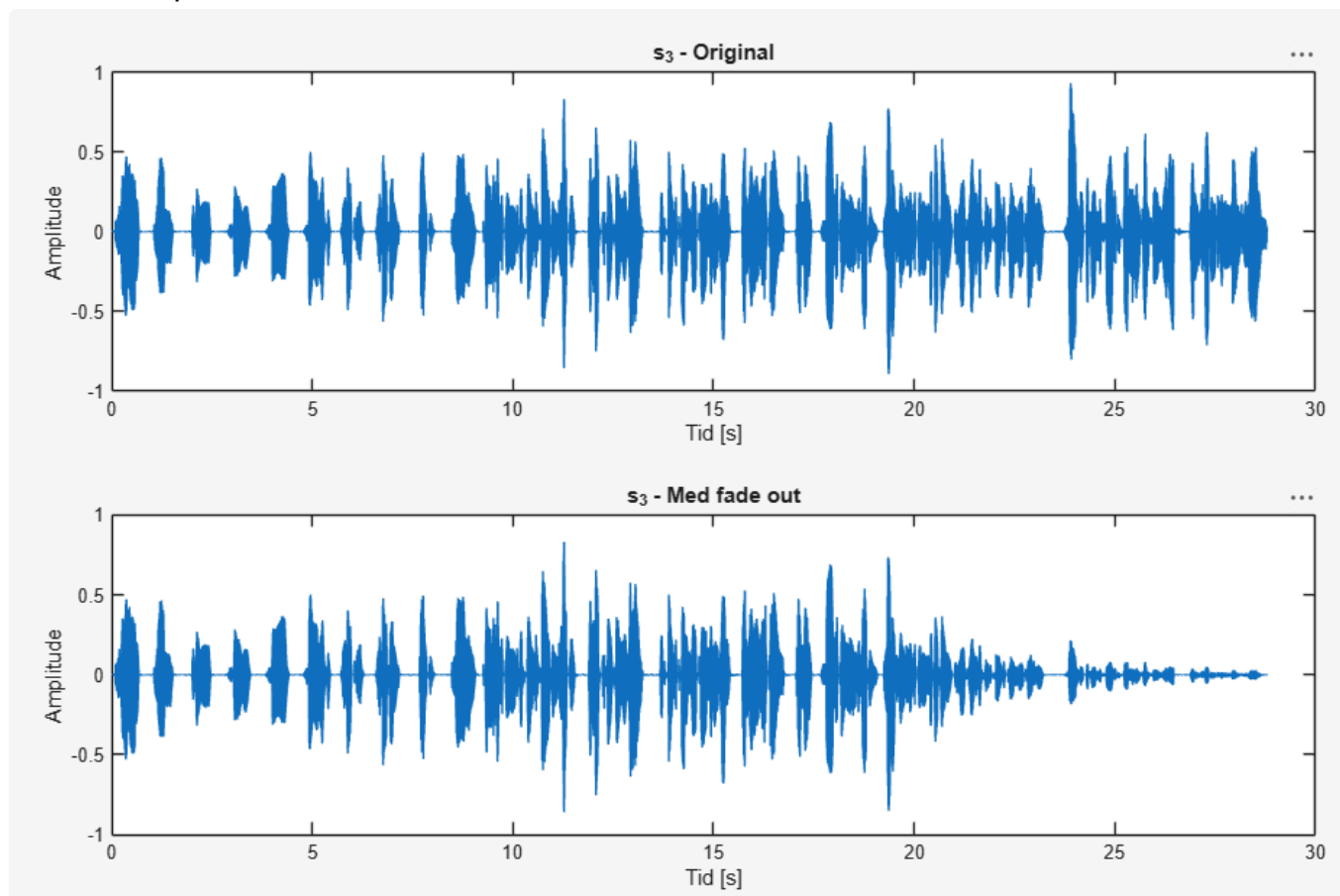
Her vil vi altså isolere α . Så formelen bliver:

$$\alpha = \frac{3 \cdot \ln(20)}{N}$$

Vi beregner det nu i matlab således:

```
x3 = data3(:,1);  
N3 = length(x3);  
N_fade = round(N3/3);  
alpha = 3*log(20)/N3;  
  
n_fade = (0:N_fade-1)';  
envelope = exp(-alpha * n_fade);  
  
s3_faded = x3;  
s3_faded(end-N_fade+1:end) = x3(end-N_fade+1:end) .* envelope;
```

Vi får disse plots:



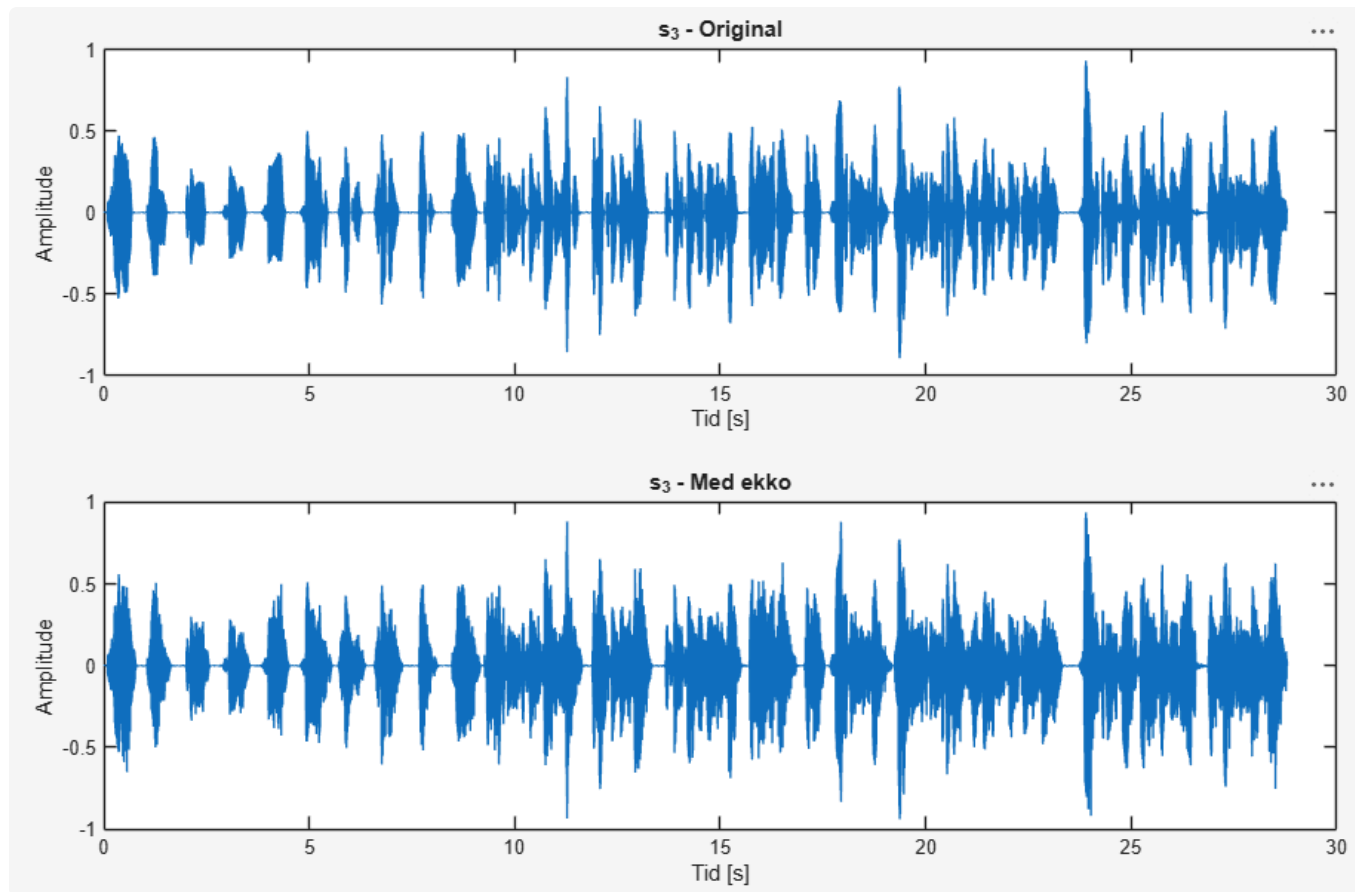
Vi kan tydeligt se det fadeout vi har fået lavet, ud fra vores plots. Det samme kan vi høre på lyden, hvor hans stemme langsomt fader ud.

10. Lav en ny udgave af s_3 , hvor der er et enkelt ekko med et delay på 100 ms.

Et ekko er bare en forsinket svagere kopi af signalet lagt oven i det originale. Vi omregner delay på 100 ms til samples ud fra vores f_s :

```
delay_ms = 100;
D = round(delay_ms/1000 * fs3);
g = 0.5;
s3_echo = x3;
s3_echo(D+1:end) = s3_echo(D+1:end) + g * x3(1:end-D);
```

Vi får disse plots:



Her kan vi se, at med ekko er det bare det originale signal lagt sammen med en svagere, forsinket version af sig selv - dvs. de samme svingninger dukker op igen lidt senere, oven i de oprindelige. Ekkoet er også tydeligt at høre når vi lytter til det.